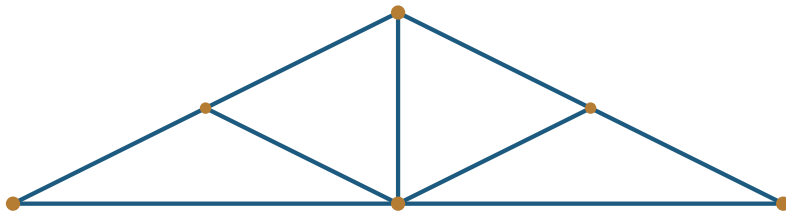


The C On-Ramp

Six short days. Just enough C to walk into Unit 1 standing up, learned by building a roof truss.



How this week works

One day per day. Read the idea, type the code on your laptop, compile and run it, then answer the quick check. Miss a day? The week has slack; just pick up where you left off. The phone app has the same six days with instant-feedback quizzes; this PDF is for reading offline.

Setup once per laptop (see the install guide): **VS Code + gcc**. Compile & run any file: `gcc file.c -o out` then `./out`.

Print a truss, run a program

A program is a list of instructions the computer runs top to bottom. Today you write three lines of shapes and watch a roof appear.

printf means "print, formatted": it puts text on the screen. **gcc** is the compiler: it turns the C you typed into a program the machine can run.

THE CODE: DAY1_HELLO_TRUSS.C

```
day1_hello_truss.c

#include <stdio.h>

int main(void) {
    printf("  /\ \  \n");
    printf(" /  \ \  \n");
    printf(" /____\ \  \n");
    printf("My first truss!\n");
    return 0;
}
```

`\n` = "new line". Each printf ends one line of the drawing.

BUILD IT

Compile: `gcc day1_hello_truss.c -o day1` then run: `./day1`. You should see the little roof, then **My first truss!**. That's the loop you repeat every day: write → compile → run.

CHECK YOURSELF

1. What does `gcc` do?

- a. Runs a website
- b. Turns your C code into a program you can run
- c. Deletes your files

b. gcc is the compiler: it translates C into a runnable program.

2. What does `printf` do?

- a. Prints text to the screen
- b. Prints paper on a printer
- c. Reads the keyboard

a. printf writes text to the screen. The **f** is for "formatted".

3. In `gcc day1.c -o day1`, what is `-o day1`?

- a. An error
- b. The name of the finished program
- c. The C version

b. `-o` names the output program. Here it's called **day1**.

Boxes that hold your numbers

A **variable** is a labelled box that holds one value. The **type** says what kind of value fits.

Three types carry almost everything: **int** for whole numbers (member count), **float** for decimals (span, pitch), and **char** for a single character (timber grade).

THE CODE: DAY2_VARIABLES.C

day2_variables.c

```
#include <stdio.h>

int main(void) {
    int    members = 7;      /* whole number */
    float  span    = 6.0;    /* metres */
    float  pitch   = 30.0;   /* roof angle */
    char   grade   = 'B';    /* one character */

    printf("Members: %d\n", members);
    printf("Span: %.1f m\n", span);
    printf("Pitch: %.0f degrees\n", pitch);
    printf("Grade: %c\n", grade);
    return 0;
}
```

The slots `%d %f %c` fill from your variables: `%d` = int, `%.1f` = float to 1 decimal, `%c` = char.

BUILD IT

`gcc day2_variables.c -o day2` then `./day2`. Then change the span to **8.5** and the grade to **'A'**, run again, watch the output change.

CHECK YOURSELF

1. Which type holds a decimal like 6.0?

- a. int
- b. float
- c. char

b. float holds decimals; int is whole numbers only.

2. `%d` in printf fills with a...

- a. Whole number (int)
- b. Decimal
- c. Single letter

a. %d = int, %f = float, %c = char.

3. `char grade = 'B';`, single quotes hold...

- a. A whole sentence
- b. Exactly one character
- c. A number

b. Single quotes = one char. Words (many chars) use double quotes.

Let the user type the span

So far the numbers were baked in. **scanf** reads what the user types and drops it into a variable.

Notice the **&** before the variable. It means "the address of": you're telling scanf *where* the box lives so it can fill it. Keep it in mind: in Unit 4 the **&** becomes **pointers**, and it'll already feel familiar.

THE CODE: DAY3_INPUT.C

day3_input.c

```
#include <stdio.h>

int main(void) {
    float span;
    printf("Enter the span in metres: ");
    scanf("%f", &span);          /* & = "put it in span" */
    printf("You entered %.2f m\n", span);
    return 0;
}
```

BUILD IT

`gcc day3_input.c -o day3` then `./day3`. Type a number, press Enter. Try a whole number and a decimal, both come back with two decimal places.

CHECK YOURSELF

1. What does **scanf** do?

- a. Prints to screen
- b. Reads what the user types
- c. Scans for viruses

b. scanf reads keyboard input into a variable.

2. Why the **&** in **scanf("%f", &span)** ?

- a. It multiplies
- b. It tells scanf WHERE to store the value
- c. It's a typo

b. **&** = "address of". It hands scanf the box's location. This idea grows into **pointers** in Unit 4.

3. **%f** reads a...

- a. Decimal number
- b. Word
- c. Whole number only

a. %f matches a float (decimal).

Ask a question, branch on the answer

A program earns its keep when it *decides*. **if** runs a block only when a test is true; **else** catches the rest.

Rule of thumb on site: a simple truss handles a moderate span. Too wide needs an engineer; too small, a single beam. Let the program say which.

THE CODE: DAY4_DECISIONS.C

day4_decisions.c

```
#include <stdio.h>

int main(void) {
    float span;
    printf("Enter the span in metres: ");
    scanf("%f", &span);

    if (span > 12.0) {
        printf("Too wide - needs an engineer.\n");
    } else if (span < 3.0) {
        printf("Very small - a single beam may do.\n");
    } else {
        printf("Standard truss territory. Good.\n");
    }
    return 0;
}
```

BUILD IT

Run it three times: try **15**, **2**, then **6**. Each should hit a different message. If one doesn't, re-read the tests; that's debugging.

CHECK YOURSELF

1. If span is 15, which message prints?

- a. Too wide - needs an engineer
- b. Standard truss territory
- c. Very small

a. $15 > 12$, so the first `if` is true and wins.

2. `else if` runs when...

- a. Always
- b. The if above was false AND its own test is true
- c. Never

b. Each branch is only reached if the ones above were false.

3. If span is 6, the output is...

- a. Too wide
- b. Very small
- c. Standard truss territory. Good.

c. 6 is not >12 and not <3 , so the `else` catches it.

Do the same job many times

A roof has many identical rafters. A **loop** repeats a block instead of you copying it. The **for** loop counts: start, keep-going test, step.

Peek ahead: `lengths[0]` is an **array**, a row of boxes. That's the whole of Unit 1, and you're already using it.

THE CODE: `DAY5_LOOPS.C`

day5_loops.c

```
#include <stdio.h>

int main(void) {
    int i, rafters = 5;
    for (i = 1; i <= rafters; i++) {
        printf("Rafter %d cut and ready\n", i);
    }

    float lengths[3] = {2.5, 3.0, 2.5}; /* a row of boxes */
    float total = 0.0;
    for (i = 0; i < 3; i++) {
        total = total + lengths[i];
    }
    printf("Total timber: %.1f m\n", total);
    return 0;
}
```

`i++` means "add 1 to i". The first loop counts 1,2,3,4,5.

BUILD IT

`gcc day5_loops.c -o day5` then `./day5`. Change **rafters** to 8, and add a 4th length; remember to change `[3]` to `[4]` and the loop's `3` to `4`.

CHECK YOURSELF

1. `for (i=1; i<=5; i++)` runs how many times?

- a. 4
- b. 5
- c. 6

b. i goes 1,2,3,4,5: five passes.

2. Across the loop, `total = total + lengths[i]` ...

- a. Adds every length together
- b. Prints each length
- c. Deletes lengths

a. It accumulates the running total, the sum of all members.

3. `lengths[0]` is...

- a. The first length (2.5)
- b. The last length
- c. The total

a. Arrays count from **0**, so `[0]` is the first box. Remember that for Unit 1.

Name a job, reuse it

A **function** is a named chunk of work. You give it inputs (arguments), it hands back a result. Write it once, call it anywhere.

A word like a member label is a **string**, stored in a **char array**. That's Unit 2, previewed. After today you've touched a little of the next six weeks already.

THE CODE: DAY6_FUNCTIONS.C

```
day6_functions.c

#include <stdio.h>

/* rough truss height from span and pitch */
float truss_height(float span, float pitch_deg) {
    float factor = pitch_deg / 100.0; /* simplified */
    return (span / 2.0) * factor;
}

int main(void) {
    char label[10] = "TC1"; /* top chord #1 */
    float h = truss_height(6.0, 30.0);
    printf("Member %s\n", label);
    printf("Approx height: %.2f m\n", h);
    return 0;
}
```

`%s` prints a string. The height math is deliberately rough; from Unit 1 on you'll make it real.

BUILD IT

Run it, then call `truss_height` a second time with a different span and print that too. One function, used twice: that's the whole point.

CHECK YOURSELF

1. A function lets you...

- Name a chunk of work and reuse it
- Slow the program down
- Only run once

a. Write the logic once, call it as many times as you like.

2. In `truss_height(6.0, 30.0)`, 6.0 and 30.0 are...

- Errors
- The inputs (arguments)
- Line numbers

b. They're the **arguments**: the values you feed the function.

3. `char label[10]` holds...

- a. One number
- b. A short word / string
- c. A whole book

b. A char array holds a string, up to 9 letters plus a hidden end-marker.

You made it. What's next.

You can now write, compile and run C, use variables, read input, make decisions, loop, and write a function. That's the whole toolkit for **Unit 1: Arrays**, where the truss cut-list becomes a real array and the build begins in earnest.